

Advanced RAG Techniques: Mathematical Foundations and Worked Examples

Companion Reference for the Agentic BI Platform – Phase 5

Agentic BI Platform Project

September 2026

Contents

1	Introduction and Purpose	3
1.1	Notation used throughout	3
2	Baseline recap: dense retrieval via cosine similarity	3
3	Maximal Marginal Relevance (MMR)	4
3.1	The problem MMR is trying to solve	4
3.2	The formula	4
3.3	Worked example	5
4	MultiQueryRetriever	6
4.1	Mechanism	6
4.2	Worked example	6
5	BM25 (Okapi BM25)	6
5.1	Why BM25 is fundamentally different from everything above	6
5.2	The full formula	7
5.3	Worked numerical example	7
6	Reciprocal Rank Fusion (RRF) and Hybrid Retrieval	8
6.1	Why not just add the two scores together?	8
6.2	The RRF formula	8
6.3	Worked example, using this project’s actual case #4	8
7	Cross-Encoder Reranking	9
7.1	Why this needs a different architecture entirely	9
7.2	Step-by-step mechanism	9
7.3	Why this is expensive, and why that expense is bounded in practice	10
7.4	Connecting this back to the project’s finding	11
8	ParentDocumentRetriever	11
9	Contextual Retrieval	12

10RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval	12
10.1 Step 1: Gaussian Mixture Models (GMM)	12
10.2 Step 2: choosing the number of clusters, K	13
10.3 Step 3: summarization and recursion	14
10.4 Worked illustration of why the tree collapsed after one level	14
11ColBERT and Late Interaction	14
11.1 Positioning ColBERT between the two extremes already covered	14
11.2 The mechanism	15
11.3 Worked toy example	15
11.4 Complexity comparison, precisely	16
11.5 Why this project could not measure it	16
12Self-Query Retrieval	16
12.1 Why this section has comparatively little arithmetic	16
12.2 The pipeline	16
12.3 Why this project could not measure it either	17
13Summary table	17
14Appendix: glossary of symbols used across sections	19

1 Introduction and Purpose

Phase 5 of the Agentic BI Platform project evaluated nine Advanced RAG techniques against a fixed retrieval quality test set built from a five-document policy/definitions/schema knowledge base. Every technique was implemented and measured; two (ColBERT, Self-Query Retrieval) were blocked by dependency conflicts before a full quality comparison could be run. This document is the missing piece: a from-first-principles explanation of **how each technique actually computes what it computes**, with worked numerical examples grounded in the project's own corpus (the restocking-fee, active-customer, and seller-region questions used throughout the evaluation), so that every technique – adopted, rejected, or blocked – can be explained and reproduced on a whiteboard, not just recalled as a name.

The document assumes you already know what each technique is *for* (that context lives in docs/decisions/decision_log.md). Its job is to answer the harder question: **what arithmetic is actually happening**, step by step, when a retriever ranks or reranks a document.

1.1 Notation used throughout

Symbol	Meaning
Q	A user's query (a string, or its embedding, by context)
D	A document or chunk
$\mathbf{q}, \mathbf{d}$	Embedding vectors for a query and a document
$\text{sim}(\mathbf{a}, \mathbf{b})$	A similarity function, usually cosine similarity
R	The full candidate set retrieved so far
S	The subset already selected (used in MMR)
k	Either "top- k results returned" or a tunable smoothing constant, depending on context – disambiguated at each use
λ	A weighting parameter between 0 and 1

2 Baseline recap: dense retrieval via cosine similarity

Every technique in this document is a modification of, or an alternative to, this baseline, so it is worth stating precisely.

A **bi-encoder** (the architecture behind text-embedding-3-small, the embedding model used throughout this project) maps a piece of text to a single fixed-length vector, independently of any other text:

$$\mathbf{q} = \text{Encoder}(Q), \quad \mathbf{d} = \text{Encoder}(D)$$

Relevance is then computed purely from the geometry of these two vectors, almost always via **cosine similarity**:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} = \frac{\sum_{i=1}^n q_i d_i}{\sqrt{\sum_{i=1}^n q_i^2} \sqrt{\sum_{i=1}^n d_i^2}}$$

This value ranges from -1 (opposite direction) to 1 (identical direction), with 0 meaning orthogonal (unrelated). FAISS’s default similarity search ranks every chunk in the index by this value against the query vector and returns the top k .

Why this is fast: every document vector $\mathbf{d}$ can be computed once, at index-build time, and stored. At query time, only $\mathbf{q}$ needs to be computed; scoring against N stored vectors is N dot products – cheap, and trivially parallelizable. This is exactly why bi-encoders scale to millions of documents, and exactly why they are the default first stage of nearly every retrieval pipeline, including every technique described below.

Why this is imprecise: the entire meaning of a passage is compressed into one vector *before* the model ever sees the query. Two chunks that are both “about returns” end up with similar vectors even if one directly answers a specific question and the other only mentions the topic in passing. Nothing in this architecture lets the model look at the query and the document *together* and reason about their relationship – that capability is exactly what cross-encoders (Reranking) and late-interaction models (ColBERT) add back in, at a computational cost.

3 Maximal Marginal Relevance (MMR)

3.1 The problem MMR is trying to solve

Plain top- k similarity search can return several near-duplicate chunks if they are all close to the query vector, wasting the retrieval budget on redundant content instead of diverse relevant content.

3.2 The formula

MMR builds the result set **greedily**, one document at a time. Given a candidate pool R (typically a wider `fetch_k` set) and an already-selected set S (initially empty), the next document chosen is:

$$\text{MMR} = \arg \max_{D_i \in R \setminus S} \left[\lambda \cdot \text{sim}_1(D_i, Q) - (1 - \lambda) \cdot \max_{D_j \in S} \text{sim}_2(D_i, D_j) \right]$$

Read this as two competing terms:

- $\text{sim}_1(D_i, Q)$: how relevant is this candidate to the query? (standard cosine similarity)
- $\max_{D_j \in S} \text{sim}_2(D_i, D_j)$: how similar is this candidate to the **most similar document already picked**? This is a redundancy penalty – it only looks at the closest existing pick, not an average.

λ controls the trade-off: $\lambda = 1$ ignores diversity entirely and behaves identically to plain top- k similarity search; $\lambda = 0$ ignores relevance entirely and just picks whatever is maximally different from what’s already chosen (which can pick things utterly unrelated to the query). The library default, and the value first tested in this project, is $\lambda = 0.5$.

3.3 Worked example

Suppose `fetch_k` = 4 candidates were retrieved for the query “*Are damaged items subject to the restocking fee?*”, with similarity-to-query scores:

Candidate	$\text{sim}_1(D_i, Q)$
A: “Eligible Reasons”	0.91
B: “Restocking Fee”	0.89
C: “Non-Returnable Categories”	0.75
D: “Seller Responsibility”	0.60

Step 1 - pick the first document. With $S = \emptyset$, the redundancy term is undefined (or zero by convention), so MMR simply picks the highest-relevance candidate: **A** (0.91).

Step 2 - pick the second document. Now $S = \{A\}$. For each remaining candidate, compute the MMR score with $\lambda = 0.5$. Suppose the similarity of each remaining candidate to A (not to the query) is: B-to-A = 0.88 (very similar – both about refund categories), C-to-A = 0.40, D-to-A = 0.30.

$$\text{MMR}(B) = 0.5(0.89) - 0.5(0.88) = 0.445 - 0.440 = 0.005$$

$$\text{MMR}(C) = 0.5(0.75) - 0.5(0.40) = 0.375 - 0.200 = 0.175$$

$$\text{MMR}(D) = 0.5(0.60) - 0.5(0.30) = 0.300 - 0.150 = 0.150$$

Despite B having the second-highest raw relevance (0.89), its heavy overlap with A already in S drags its MMR score down to nearly zero. **C is selected next** (0.175 is the highest), even though it was the *least* query-relevant of the three original alternatives.

This is precisely the mechanism identified as the project’s real failure mode: when a corpus’s chunks from the same document are **not actually redundant** with each other (as this project’s turned out to be – each markdown section covers a genuinely distinct rule), the penalty term $\max_{D_j \in S} \text{sim}_2(D_i, D_j)$ is measuring something that isn’t really there, and it ends up demoting a second useful chunk from the correct document in favor of an unrelated one from a different document, purely to satisfy the diversity term.

4 MultiQueryRetriever

4.1 Mechanism

There is no new similarity formula here – the innovation is entirely in what gets *searched for*. Given an original question Q , an LLM generates n paraphrased variants $Q_1, Q_2, \dots, Q_n$. Each variant is run through the **same** bi-encoder similarity search described above, producing n separate top- k result lists $R_1, R_2, \dots, R_n$. These lists are unioned and deduplicated (by chunk identity, not by content similarity):

$$R_{\text{final}} = R_1 \cup R_2 \cup \dots \cup R_n$$

There is no re-scoring or re-ranking step across variants – a chunk that appears in R_1 and also happens to appear in R_3 is simply included once, and no combined score is computed.

4.2 Worked example

Query: “What counts as an active customer?” An LLM might generate:

- Q_1 : “What is the definition of an active customer?”
- Q_2 : “Under what conditions is a customer considered active?”
- Q_3 : “How does the platform classify a customer as active or inactive?”

Each is embedded and searched independently. If Q_1 ’s top-4 already contains the correct `kpi_definitions.md` chunk at rank 1 (which, empirically, it did in this project – `text-embedding-3-small` already handles this exact paraphrase gap), then Q_2 and Q_3 add cost (two extra embedding calls, two extra similarity searches) without adding information the first query didn’t already surface. This is precisely why the technique measured as **cost without benefit** on this corpus: its entire value proposition is bridging a vocabulary gap the base embedding model already bridges unaided.

5 BM25 (Okapi BM25)

5.1 Why BM25 is fundamentally different from everything above

Every technique so far scores a document by geometric closeness of **learned, continuous** embedding vectors. BM25 has no learned model, no embedding, and no neural network at all. It is a formula over **raw word counts**, refined from decades of information-retrieval research predating modern NLP. Understanding it is what makes “why can hybrid search recover a case that dense search missed” a mechanically precise question rather than a vague intuition.

5.2 The full formula

For a query Q containing terms $q_1, q_2, \dots, q_m$ and a document D :

$$\text{score}(D, Q) = \sum_{i=1}^m \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

Breaking this into its three real components:

1. Term frequency saturation. $f(q_i, D)$ is the raw count of term q_i in document D . The fraction $\frac{f \cdot (k_1 + 1)}{f + k_1}$ is a *saturating* function of f : it grows quickly at first but flattens out, because a document mentioning “restocking” 10 times is not really 10 times more about restocking than one that mentions it once. k_1 (typically 1.2-2.0) controls how quickly this saturation kicks in.

2. Length normalization. The term $\left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)$ penalizes documents longer than the corpus average (avgdl), since a long document is statistically more likely to contain any given term by chance alone, purely from having more words. b (typically 0.75) controls how strongly length is penalized; $b = 0$ disables length normalization entirely.

3. Inverse document frequency (IDF). This is the term that gives BM25 its “rare words matter more” behavior:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

where N is the total number of documents (chunks) in the corpus and $n(q_i)$ is the number of chunks containing term q_i at least once. A term appearing in almost every chunk (like “the,” “customer,” or, in this corpus, “return”) gets a **low** IDF weight – it isn’t discriminating between chunks. A term appearing in only one or two chunks (like “restocking” or a specific number like “20”) gets a **high** IDF weight, because finding it is strong evidence about which chunk is relevant.

5.3 Worked numerical example

Suppose across this project’s 44 chunks, the term “restocking” appears in exactly 2 chunks, while the term “customer” appears in 30 chunks ($N = 44$).

$$\text{IDF}(\text{“restocking”}) = \ln \left(\frac{44 - 2 + 0.5}{2 + 0.5} + 1 \right) = \ln \left(\frac{42.5}{2.5} + 1 \right) = \ln(18.0) \approx 2.89$$

$$\text{IDF}(\text{“customer”}) = \ln \left(\frac{44 - 30 + 0.5}{30 + 0.5} + 1 \right) = \ln \left(\frac{14.5}{30.5} + 1 \right) = \ln(1.475) \approx 0.39$$

The term “restocking” contributes roughly **7.4 times** more weight per occurrence than “customer” does. This is the exact mechanism by which BM25 is, in principle, well-suited to a query like “*what is the restocking fee percentage*” – the rare, highly specific term dominates the score, regardless of how many times more common words like “customer” or “return” appear elsewhere.

Why it still underperformed on this project’s k=2 test (case #2, “7 calendar days”): BM25 scores whole documents/chunks by summed term contributions across *all* query terms, not by whether an exact phrase appears. The query “how long does a customer have to request a return” contains no term as rare and distinctive as “restocking” – every individual word (“customer,” “return,” “long,” “request”) is common across the corpus, so no single term’s IDF was high enough to pull the correct chunk above competing ones once only the top 2 were kept. This is a real, mechanistically explainable limitation, not a bug: **BM25 rewards rare terms, but a query built entirely from common words gives it nothing to discriminate on.**

6 Reciprocal Rank Fusion (RRF) and Hybrid Retrieval

6.1 Why not just add the two scores together?

A natural first instinct is to combine a BM25 score and a cosine similarity score directly – but they live on **completely incompatible scales**. Cosine similarity is bounded in $[-1, 1]$; a BM25 score is an unbounded sum of log-IDF-weighted saturating fractions, which might be 0.3 for one query and 14.7 for another depending on term rarity. Averaging or summing these two numbers directly would let whichever happens to have larger numbers dominate, for reasons that have nothing to do with actual relevance.

6.2 The RRF formula

Reciprocal Rank Fusion sidesteps the scale problem entirely by throwing away the raw scores and using only **rank position**:

$$\text{RRF}(D) = \sum_{r \in \text{retrievers}} \frac{1}{k + \text{rank}_r(D)}$$

where $\text{rank}_r(D)$ is the 1-indexed position of document D in retriever r ’s ranked list (or ∞ , contributing 0, if D never appears in that list at all), and k is a smoothing constant (conventionally $k = 60$, chosen empirically in the original RRF paper to avoid the top-ranked document dominating too heavily).

6.3 Worked example, using this project’s actual case #4

Recall the empirical finding: for the query “*Is a customer still enterprise if they haven’t ordered in a while?*”, dense search and BM25 ranked the two relevant documents in **opposite order**.

Document	Dense rank	BM25 rank
customer_segmentation.md (chunk X)	1	3
kpi_definitions.md (chunk Y)	4	1

Using $k = 60$:

$$\text{RRF}(\text{chunk X}) = \frac{1}{60 + 1} + \frac{1}{60 + 3} = 0.01639 + 0.01587 = 0.03226$$

$$\text{RRF}(\text{chunk Y}) = \frac{1}{60 + 4} + \frac{1}{60 + 1} = 0.01563 + 0.01639 = 0.03202$$

Both scores end up **very close to each other** (0.03226 vs 0.03202) – because each document is ranked highly by exactly one retriever and mediocly by the other, RRF gives them near-equal combined weight. Under *either* single retriever alone, one of these two chunks would have been pushed out of a $k=2$ window entirely (chunk Y is rank 4 under dense search alone; chunk X is rank 3 under BM25 alone). Fused, both survive into a small top- k window. This is the exact, arithmetic reason hybrid retrieval was the only technique in this project to beat plain dense search’s recall at a reduced k : **it does not need either individual ranker to be good at this query – it only needs the two rankers to be wrong in different, uncorrelated ways.**

This last point is worth restating precisely, since it is the single most reusable insight from this whole evaluation: **RRF’s value is a function of how *independent* the errors of its component rankers are, not of how strong either ranker is individually.** A corpus where dense and lexical search tend to agree (rank documents similarly) gains little from fusing them; a corpus where they disagree – as this one’s case 4 did – is exactly where fusion earns its cost.

7 Cross-Encoder Reranking

7.1 Why this needs a different architecture entirely

A bi-encoder never lets the query and document “meet” inside the neural network – they are encoded completely separately, and only compared afterward via cosine similarity. A **cross-encoder** removes this separation entirely: the query and document are concatenated into a **single input sequence** and passed through the transformer *together*, so every token of the query can directly attend to every token of the document (and vice versa) at every layer.

7.2 Step-by-step mechanism

Step 1 - construct the joint input. For query Q and document D , build a single token sequence in the BERT-style format:

$$[\text{CLS}] \ Q_1, Q_2, \dots, Q_n \ [\text{SEP}] \ D_1, D_2, \dots, D_m \ [\text{SEP}]$$

Step 2 - self-attention over the whole sequence. Each transformer layer computes multi-head self-attention over this *entire* joint sequence at once. For a single attention head, given the layer’s input matrix X (one row per token, whether it came from Q or D), the model learns three projection matrices W_Q, W_K, W_V and computes:

$$\text{Attention}(X) = \text{softmax}\left(\frac{XW_Q(XW_K)^\top}{\sqrt{d_k}}\right) XW_V$$

Critically, X here contains rows from **both** Q and D together. A query token’s row can attend to (and be directly influenced by) any document token’s row in the same softmax computation – this is the literal meaning of “cross-attention” in cross-encoder, and it is precisely the capability a bi-encoder’s two independent forward passes cannot provide, since a bi-encoder never constructs a joint sequence at all. This attention computation repeats across every layer of the network (12, 24, or more layers depending on model size), so the representation of every token is iteratively refined by information from tokens in the *other* sequence, many times over.

Step 3 - pool to a single relevance score. After the final layer, the model takes the final hidden state of the special [CLS] token (which, by construction and training, has absorbed a summary of the entire joint sequence) and passes it through a small learned linear layer (and typically a sigmoid) to produce one scalar:

$$\text{score}(Q, D) = \sigma(\mathbf{w}^\top \mathbf{h}_{[\text{CLS}]} + b)$$

where $\mathbf{h}_{[\text{CLS}]}$ is the final-layer [CLS] vector, and $\mathbf{w}, b$ are trained parameters. This scalar is the reranker’s relevance score – not a cosine similarity, not a probability derived from two independently-computed vectors, but the output of a model that saw the query and document jointly and was trained (via labeled query-document relevance pairs, using a loss such as pairwise ranking loss or binary cross-entropy) specifically to predict relevance from that joint view.

7.3 Why this is expensive, and why that expense is bounded in practice

A cross-encoder must run a **full transformer forward pass per (query, document) pair** – there is no way to precompute document representations offline, because the representation of a document token depends on which query it is being compared against. This is why reranking is always applied to a small candidate set (`fetch_k`, e.g. 10) returned by a cheap first-stage retriever, never to an entire index: scoring 10 candidates means 10 forward passes; scoring 30,000 chunks this way would be computationally prohibitive for a live query.

7.4 Connecting this back to the project’s finding

This project’s Reranking evaluation showed the cross-encoder correctly identifying that `pricing_and_freight_policy.md`’s rank-3 hit (under plain dense search) came from an unrelated section, while `data_dictionary.md` contained a more directly relevant sentence. This is exactly the kind of judgment a bi-encoder’s single-vector-per-chunk architecture is structurally unable to make with the same precision: the cross-encoder examined the *specific words of the query* against the *specific words of each candidate chunk, jointly*, rather than comparing two independently-compressed summary vectors.

8 ParentDocumentRetriever

There is no new similarity mathematics in this technique – it reuses the same bi-encoder cosine similarity search described in the baseline section, applied to small **child** chunks. The mechanism worth understanding precisely is the **indexing and lookup structure**, since that structure is what caused the measured regression.

Index-time: each raw document is split twice, at two different granularities – a `parent_splitter` (large, e.g. 1200 characters) and a `child_splitter` (small, e.g. 250 characters) nested inside each parent. Only the **child** chunks are embedded and stored in the vector index. Each child is tagged with a reference to which parent it belongs to, and the parents themselves are stored separately in a document store (not a vector index – they are never embedded).

Query-time: similarity search runs exactly as in the baseline, against the child vectors. Once the top- k **child** chunks are identified, the retriever looks up each one’s parent ID and returns the **deduplicated set of parent documents** – if child chunks #3 and #7 both happen to belong to parent document P2, only one copy of P2 is returned, not two.

Why this caused a measured regression: the parameter k still controls how many *child* chunks are drawn from the vector index, not how many *distinct parents* end up in the final result. If the top- k child matches cluster heavily within one dominant parent document (exactly the scenario already established for this corpus’s case 4, where `customer_segmentation.md` out-competes `kpi_definitions.md`), the deduplication step can silently collapse what should have been 2 useful results into 1, because both selected children happened to share a parent. Formally, if k children are drawn and they map to $p \leq k$ distinct parents, the number of documents actually returned is p , which can be strictly less than k – an effective, uncontrolled reduction in retrieval width that this project’s evaluation measured directly (recall dropping to 7/9 at a nominal $k = 2$, worse than plain similarity search’s 8/9 at the identical nominal k).

9 Contextual Retrieval

Like ParentDocumentRetriever, this technique introduces no new similarity formula – the underlying search is still baseline cosine similarity. Its entire contribution happens **before** embedding, at chunk-preparation time, and is best understood as a text-transformation step with a measurable cost, not a new piece of retrieval mathematics.

The transformation: for each chunk D (drawn from full source document $\mathcal{D}$), an LLM call generates a short situating description $C = \text{LLM}(\mathcal{D}, D)$ – a sentence or two stating which section this chunk is from and what broader rule it depends on. The chunk actually embedded and indexed is not D but the concatenation:

$$D' = C \parallel D$$

where $\parallel$ denotes text concatenation. The embedding model then produces $\mathbf{d}' = \text{Encoder}(D')$ instead of $\mathbf{d} = \text{Encoder}(D)$, and similarity search proceeds exactly as in the baseline, but against this augmented vector.

Cost, stated precisely: this requires exactly one LLM call per chunk, at index-build time (not per query). For this project’s 44 chunks, that is 44 LLM calls to build the index once; rebuilding the index (e.g. after adding a new document) repeats all 44 calls, plus one per any new chunk.

Why it is non-deterministic in a way none of the other techniques are: every technique above this section computes a fixed function of fixed inputs – the same query against the same index always returns the same ranking. Contextual Retrieval’s index itself depends on LLM sampling (temperature, even at 0.0, does not guarantee bit-identical output across all providers and model versions), so C , and therefore $\mathbf{d}'$, and therefore the *entire index’s* geometry, can differ subtly between two builds from identical source documents. This project’s evaluation observed exactly this: case 4 failed on one build and passed on an identical rebuild, a property no deterministic technique (MMR, Reranking, ParentDocumentRetriever, BM25/Hybrid) can exhibit.

10 RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval

This is the most mathematically involved technique evaluated in this project, because – unlike everything above, which reuses a similarity search primitive – RAPTOR requires an actual **unsupervised clustering algorithm** as a building block. This section explains that algorithm in full before returning to how RAPTOR uses it.

10.1 Step 1: Gaussian Mixture Models (GMM)

A Gaussian Mixture Model assumes that the data (here: chunk embedding vectors) was generated by a mixture of K multivariate normal distributions. The overall probability of observing a point x is:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

where π_k is the mixing weight of cluster k (with $\sum_k \pi_k = 1$), μ_k is that cluster's mean vector, and Σ_k is its covariance matrix. Unlike k-means (which assigns each point to exactly one cluster), a GMM produces a **soft assignment** – the probability that point x belongs to cluster k , called its *responsibility*:

$$\gamma_k(x) = \frac{\pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x \mid \mu_j, \Sigma_j)}$$

This is exactly why RAPTOR (in its full published form) allows a single chunk to belong to more than one cluster: if $\gamma_k(x)$ is high for two different clusters k , the chunk can be included in both clusters' summaries.

Fitting the model (Expectation-Maximization): since neither the cluster assignments nor the parameters (π_k, μ_k, Σ_k) are known in advance, they are estimated iteratively:

- **E-step:** given the current parameter estimates, compute $\gamma_k(x)$ for every point (as above).
- **M-step:** given the current responsibilities, re-estimate the parameters as responsibility-weighted statistics:

$$\mu_k = \frac{\sum_x \gamma_k(x) x}{\sum_x \gamma_k(x)}, \quad \pi_k = \frac{1}{n} \sum_x \gamma_k(x)$$

These two steps alternate until the parameters converge (the log-likelihood of the data stops improving meaningfully).

10.2 Step 2: choosing the number of clusters, K

K is not fixed in advance – the algorithm implemented in this project's `_cluster_embeddings` fits several candidate values of K and picks the one minimizing the **Bayesian Information Criterion**:

$$\text{BIC} = -2 \ln(\hat{L}) + p \ln(n)$$

where $\hat{L}$ is the model's maximized likelihood of the observed data, p is the number of free parameters in the model (which grows with K : more clusters means more means and covariances to fit), and n is the number of data points. The $-2 \ln(\hat{L})$ term rewards a better fit to the data; the $p \ln(n)$ term penalizes model complexity. BIC therefore formalizes exactly the tradeoff you would reason about informally: adding more clusters always improves raw fit, but each additional cluster must “earn” its

added complexity by improving the fit enough to outweigh the penalty. **This is precisely the mechanism by which this project’s implementation, given only 44 chunks across 5 documents, converged on a minimal $K = 2$** – with so little data relative to embedding dimensionality, BIC could not justify the added parameters a $K = 4$ or $K = 5$ model would require, even though there are 5 genuinely distinct source documents.

10.3 Step 3: summarization and recursion

Once chunks are clustered, each cluster’s member chunks are concatenated and passed to an LLM with a summarization prompt, producing one new “summary node” per cluster. These summary nodes are then embedded (using the same bi-encoder as everything else) and become the input to another round of the same GMM clustering process – recursively, until clustering “collapses” (too few nodes remain to meaningfully split, or the optimal K found is 1).

10.4 Worked illustration of why the tree collapsed after one level

With 44 leaf chunks and $K = 2$ selected by BIC, the algorithm produced 2 summary nodes. To recurse a second time, the algorithm needs enough summary nodes to plausibly cluster into multiple *groups of summaries* – but with only 2 summary nodes total, there is no meaningful way to split 2 items into more than 2 singleton groups, so the recursion halts immediately (as this project’s implementation explicitly checks: “stopping ... too few nodes to cluster meaningfully”). RAPTOR’s genuine value proposition – letting a broad thematic query match a high-level summary while a specific-fact query matches a low-level chunk – requires enough underlying content that **several distinct, coherent themes** exist *within* what would otherwise be a single document’s worth of material. A five-document corpus, where each document is already a natural, readable, single-topic unit, does not have that structure to discover; RAPTOR’s clustering machinery is not wrong, it simply has nothing non-trivial to find.

11 ColBERT and Late Interaction

11.1 Positioning ColBERT between the two extremes already covered

By this point, two architectures have been described, sitting at opposite ends of a cost/precision tradeoff:

- **Bi-encoder** (baseline): one vector per document, computed once, compared via a single cosine similarity. Cheap. Cannot see the query and document together.
- **Cross-encoder** (Reranking): the query and document are fused into one sequence and jointly processed by every layer of a transformer. Expensive (a full forward pass per candidate). Sees everything together, at maximum resolution.

Late interaction (ColBERT) is a deliberate middle point: it keeps per-token granularity (unlike a bi-encoder’s single pooled vector) but still encodes the query and doc-

ument **independently** (unlike a cross-encoder’s joint encoding), and only lets them “interact” via a cheap similarity computation *after* encoding.

11.2 The mechanism

Step 1 - per-token encoding, independently. For a query with tokens $Q_1, \dots, Q_n$ and a document with tokens $D_1, \dots, D_m$, a BERT-like encoder produces **one vector per token**, not one vector for the whole sequence:

$$\mathbf{q}_1, \dots, \mathbf{q}_n = \text{Encoder}(Q_1, \dots, Q_n), \quad \mathbf{d}_1, \dots, \mathbf{d}_m = \text{Encoder}(D_1, \dots, D_m)$$

Each $\mathbf{q}_i$ and $\mathbf{d}_j$ is typically L2-normalized. Critically, **the document’s token vectors do not depend on the query at all** - they can be computed once, offline, exactly like a bi-encoder’s document vectors, and stored (indexed) in advance.

Step 2 - MaxSim: late interaction at query time. For every query token, find its single best-matching document token, by plain dot product (equivalent to cosine similarity, since vectors are normalized):

$$\text{MaxSim}(\mathbf{q}_i, D) = \max_{j=1, \dots, m} \mathbf{q}_i \cdot \mathbf{d}_j$$

Step 3 - sum over all query tokens to get the final relevance score:

$$\text{score}(Q, D) = \sum_{i=1}^n \text{MaxSim}(\mathbf{q}_i, D) = \sum_{i=1}^n \max_{j=1, \dots, m} \mathbf{q}_i \cdot \mathbf{d}_j$$

11.3 Worked toy example

Suppose the query is “restocking fee” (2 tokens after tokenization, for simplicity) and a candidate chunk has 5 tokens. Suppose the per-token dot products (already computed) look like this:

	d_1 = “Restocking”	d_2 = “fee”	d_3 = “For”	d_4 = “only”	d_5 = “10%”
q_1 = “restocking”	0.95	0.40	0.10	0.05	0.20
q_2 = “fee”	0.35	0.91	0.05	0.02	0.30

$$\text{MaxSim}(q_1, D) = \max(0.95, 0.40, 0.10, 0.05, 0.20) = 0.95$$

$$\text{MaxSim}(q_2, D) = \max(0.35, 0.91, 0.05, 0.02, 0.30) = 0.91$$

$$\text{score}(Q, D) = 0.95 + 0.91 = 1.86$$

Notice what this preserves that a bi-encoder would lose: token “restocking” finds its near-perfect match at d_1 , and token “fee” finds its own near-perfect match at d_2 ,

independently – each query token gets credit for its single best point of alignment in the document, rather than the whole query and whole document being smashed into one vector each and compared once. This is why late-interaction models tend to outperform bi-encoders on precise term-level matching (the same weakness BM25 targets, but here captured through learned semantic vectors instead of raw word-overlap statistics) while remaining far cheaper than a full cross-encoder, since only the token vectors need computing per candidate at query time (a MaxSim computation over $n \times m$ dot products, typically far cheaper than $n + m$ tokens passing through dozens of full transformer layers together).

11.4 Complexity comparison, precisely

Architecture	Document representation	Query-time cost per candidate	Sees query+doc jointly?
Bi-encoder	1 vector	1 dot product	No
ColBERT (late interaction)	m vectors (1 per token)	$n \times m$ dot products	Only via MaxSim, post-hoc
Cross-encoder	N/A (recomputed per query)	1 full transformer forward pass	Yes, at every layer

11.5 Why this project could not measure it

This project’s ColBERT integration attempt (via the ragatouille library) was blocked entirely by dependency conflicts before any of the above computation ever ran – the mechanism described here is accurate to how ColBERT works in general, but no numbers specific to this corpus exist, since the index was never successfully built. This is the reason the decision log records this technique as “not evaluated,” rather than “evaluated and rejected” – an important distinction, since the former says nothing about whether the technique would have helped.

12 Self-Query Retrieval

12.1 Why this section has comparatively little arithmetic

Every technique above modifies **how relevance is scored**. Self-Query Retrieval modifies **which chunks are even considered candidates**, before any relevance scoring happens at all. Its core mechanism is a parsing and translation problem, not a numerical one – worth understanding precisely so it is not mistaken for “just another ranking formula.”

12.2 The pipeline

Step 1 - schema declaration. The developer declares, in advance, which meta-data fields exist and what they mean (this project’s `document_type`, `section_title`, `mentions_segment` fields, each described via an `AttributeInfo` object).

Step 2 - structured query construction. Given the user’s natural language question and the declared schema, an LLM is prompted to produce a **structured query**: an internal representation with two parts -

- a (possibly rewritten) semantic query string, to be embedded and compared via ordinary cosine similarity, exactly as in the baseline, and
- a **filter expression**, built from a small internal grammar of comparators (eq, ne, gt, lt, in, contain, ...) and logical operators (and, or, not), e.g. (informally):

Filter = (document_type = "definitions")

Step 3 - translation. This abstract filter expression is not yet in any particular vector database’s native filter syntax. A **translator** (implemented as a “visitor” over the filter expression tree, one implementation per supported vector store - Pinecone, Chroma, FAISS, and many others) walks the expression and emits the vector store’s actual native filter predicate.

Step 4 - filtered similarity search. Only *after* translation does anything resembling the baseline’s mathematics occur: cosine similarity search runs exactly as before, but restricted to the subset of the index satisfying the translated filter - formally, similarity is computed only over $\{D \in \text{Index} : \text{Filter}(D) = \text{true}\}$, rather than the full index.

12.3 Why this project could not measure it either

SelfQueryRetriever’s implementation eagerly imports translator classes for every vector store it supports (Pinecone, Chroma, Databricks, DeepLake, and roughly a dozen others), purely so it can perform an `isinstance` check and select the correct one for whichever vector store is actually in use (FAISS, in this project). Several of these translator dependencies had been deprecated out of the installed library version, causing the entire import to fail before Step 1 above could even begin - again, “not evaluated” rather than “evaluated and rejected.” The underlying filter-then-search idea, and the `document_type` / `section_title` / `mentions_segment` metadata fields built to support it, remain valid and were independently confirmed correct via manual content inspection; only the automatic LLM-driven filter-construction layer was blocked.

13 Summary table

Technique	New math introduced	Where the math runs	Determinism
Baseline (dense)	Cosine similarity	Query-time, per candidate	Deterministic
MMR	Greedy relevance-minus-redundancy objective	Query-time, iterative	Deterministic

Technique	New math introduced	Where the math runs	Determinism
MultiQueryRetriever	None (reuses baseline, multiple times)	Query-time (LLM + N searches)	Non-deterministic (LLM-generated variants)
BM25	Term-frequency saturation + IDF	Query-time, per candidate	Deterministic
RRF / Hybrid	Reciprocal-rank sum across rankers	Query-time, post-ranking	Deterministic (given deterministic rankers)
Cross-Encoder Reranking	Joint transformer self-attention + learned scoring head	Query-time, per candidate	Deterministic (given fixed model weights)
ParentDocumentRetriever	None (reuses baseline on child chunks)	Index-time (structure) + query-time (search)	Deterministic
Contextual Retrieval	None (reuses baseline on augmented text)	Index-time (LLM call per chunk)	Non-deterministic (LLM generation)
RAPTOR	Gaussian Mixture Models, EM, BIC	Index-time (clustering + summarization)	Non-deterministic (LLM summaries; GMM fit is deterministic given a fixed random seed)
ColBERT	Per-token encoding + MaxSim late interaction	Index-time (token vectors) + query-time (MaxSim)	Deterministic (not measured in this project)
Self-Query	None (filter-then-baseline-search)	Query-time (LLM parses filter, then baseline search)	Non-deterministic (LLM-constructed filter; not measured in this project)

14 Appendix: glossary of symbols used across sections

Symbol	Used in	Meaning
$\mathbf{q}, \mathbf{d}$	Baseline, ColBERT	Embedding vectors (whole-text or per-token)
λ	MMR	Relevance/diversity trade-off weight
S, R	MMR	Selected set / candidate pool
k_1, b	BM25	Term-frequency saturation rate / length normalization strength
$N, n(q_i)$	BM25	Total documents / documents containing term q_i
$f(q_i, D)$	BM25	Raw count of term q_i in document D
avgdL	BM25	Average document length across the corpus
k (as in RRF)	RRF	Smoothing constant (conventionally 60) – unrelated to BM25’s k_1
$\text{rank}_r(D)$	RRF	Position of document D in retriever r ’s ranked list
W_Q, W_K, W_V	Cross-Encoder	Learned attention projection matrices
$\mathbf{h}_{[CLS]}$	Cross-Encoder	Final hidden state of the [CLS] token
π_k, μ_k, Σ_k	RAPTOR (GMM)	Cluster mixing weight, mean, covariance
$\gamma_k(x)$	RAPTOR (GMM)	Soft cluster responsibility
$\hat{L}, p, n$	RAPTOR (BIC)	Maximized likelihood, parameter count, sample count
MaxSim	ColBERT	Per-query-token best match against document tokens